Chapter 13. Resource Requirements, Limits, and Quotas

A workload executed in Pods will consume a certain amount of resources (e.g., CPU and memory). You should define resource requirements for those applications. On a container level, you can define a minimum amount of resources needed to run the application, as well as the maximum amount of resources the application is allowed to consume. Application developers should determine the right sizing with load tests or at runtime by monitoring the resource consumption.

RESOURCE UNITS IN KUBERNETES

Kubernetes measures CPU resources in millicores (m), also called millicpu, and memory resources in bytes. That's why you might see resources defined as 600 m or 100 Mi. For a deep dive on those resource units, it's worth cross-referencing "Resource units in Kubernetes" in the official documentation.

COVERAGE OF CURRICULUM OBJECTIVES

This chapter addresses the following curriculum objective:

- Configure Pod admission and scheduling (limits, node affinity, etc.)

Kubernetes administrators can put measures in place to enforce the use of available resource capacity. This chapter discusses two Kubernetes primitives in this realm: the *ResourceQuota* and the *LimitRange*. The ResourceQuota defines aggregate resource constraints on a namespace level. A LimitRange is a policy that constrains or defaults the resource allocations for a single object of a specific type (such as for a Pod or a PersistentVolumeClaim).

Working with Resource Requirements

It's recommended practice that you specify resource requests and limits for every container. Determining those resource expectations is not al-

ways easy, specifically for applications that haven't been exercised in a production environment yet. Load testing the application early during the development cycle can help with analyzing the resource needs. Further adjustments can be made by monitoring the application's resource consumption after deploying it to the cluster.

Quality of Service (QoS) classes in Kubernetes automatically categorize Pods into Guaranteed, Burstable, or BestEffort tiers based on their resource requests and limits, determining their eviction priority when nodes experience resource pressure. While understanding QoS is valuable for production workloads, it's not explicitly covered in the CKA exam, which focuses more on practical resource management and Pod scheduling rather than the underlying eviction priorities.

Defining Container Resource Requests

One metric that comes into play for workload scheduling is the *resource request* defined by the containers in a Pod. Commonly used resources that can be specified are CPU and memory. The scheduler ensures that the node's resource capacity can fulfill the resource requirements of the Pod. More specifically, the scheduler determines the sum of resource requests per resource type across all containers defined in the Pod and compares them with the node's available resources.

Each container in a Pod can define its own resource requests. Table 13-1 describes the available options, including an example value.

Table 13-1. Options for resource requests

YAML attribute	Description	Example value
`spec.containers[].resources.requests.cpu`	CPU resource type	`500m` (five hundred millicpu)
`spec.containers[].resources.requests.memory`	Memory resource type	`64Mi` (2^26 bytes)
`spec.containers[].resources.requests.hugepages-<size>`	Huge page resource type	`hugepages-2Mi: 60Mi`
`spec.containers[].resources.requests.ephemeral-storage`	Ephemeral storage resource type	`4Gi`

To clarify the uses of these resource requests, we'll look at an example definition. The Pod YAML manifest shown in Example 13-1 defines two con-

tainers, each with its own resource requests. Any node that is allowed to run the Pod needs to be able to support a minimum memory capacity of 320 Mi and 1250 m CPU, the sum of resources across both containers.

Example 13-1. Setting container resource requests

```
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "256Mi"
        cpu: "1"
  - name: ambassador
    image: bmuschko/nodejs-ambassador:1.0.0
    ports:
    - containerPort: 8081
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
```

It's certainly possible that a Pod cannot be scheduled due to insufficient resources available on the nodes. In those cases, the event log of the Pod will indicate this situation with the reasons `PodExceedsFreeCPU` or `PodExceedsFreeMemory`. For more information on how to troubleshoot and resolve this situation, see the relevant section in the documentation.

Defining Container Resource Limits

Another metric you can set for a container is the *resource limits*. Resource limits ensure that the container cannot consume more than the allotted resource amounts. For example, you could express that the application running in the container should be constrained to 1,000 m of CPU and 512 Mi of memory.

Depending on the container runtime used by the cluster, exceeding any of the allowed resource limits results in a termination of the application process running in the container or results in the system preventing the

allocation of resources beyond the limits. For an in-depth discussion on how resource limits are treated by the container runtime Docker Engine, see the documentation.

Table 13-2 describes the available options, including an example value.

Table 13-2. Options for resource limits

YAML attribute	Description	Example value
`spec.containers[].resources.limits.cpu`	CPU resource type	`500m` (500 millicpu)
`spec.containers[].resources.limits.memory`	Memory resource type	`64Mi` (2^26 bytes)
`spec.containers[].resources.limits.hugepages-<size>`	Huge page resource type	`hugepages-2Mi: 60Mi`
`spec.containers[].resources.limits.ephemeral-storage`	Ephemeral storage resource type	`4Gi`

Example 13-2 shows the definition of limits in action. Here, the container named `business-app` cannot use more than 256 Mi of memory. The container named `ambassador` defines a limit of 64 Mi of memory.

Example 13-2. Setting container resource limits

```
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      limits:
        memory: "256Mi"
  - name: ambassador
    image: bmuschko/nodejs-ambassador:1.0.0
    ports:
    - containerPort: 8081
    resources:
      limits:
        memory: "64Mi"
```

Defining Container Resource Requests and Limits

To provide Kubernetes with the full picture of your application's resource expectations, you must specify resource requests and limits for every container. Example 13-3 combines resource requests and limits in a single YAML manifest.

Example 13-3. Setting container resource requests and limits

```
apiVersion: v1
kind: Pod
metadata:
  name: rate-limiter
spec:
  containers:
  - name: business-app
    image: bmuschko/nodejs-business-app:1.0.0
    ports:
    - containerPort: 8080
    resources:
      requests:
        memory: "256Mi"
        cpu: "1"
      limits:
        memory: "256Mi"
  - name: ambassador
    image: bmuschko/nodejs-ambassador:1.0.0
    ports:
    - containerPort: 8081
    resources:
      requests:
        memory: "64Mi"
        cpu: "250m"
      limits:
        memory: "64Mi"
```

Assigning static container resource requirements is an approximation process. You want to maximize an efficient utilization of resources in your Kubernetes cluster. Unfortunately, the Kubernetes documentation does not offer a lot of guidance on best practices. The blog post "For the Love of God, Stop Using CPU Limits on Kubernetes" by Natan Yellin provides the following guidance:

- Always define memory requests.
- Always define memory limits.
- Always set your memory requests equal to your limit.
- Always define CPU requests.

- Do not use CPU limits.

After launching your application to production, you still need to monitor your application resource consumption patterns. Review resource consumption at runtime and keep track of actual scheduling behavior and potential undesired behaviors once the application receives load. Finding a happy medium can be challenging. Projects like Goldilocks and KRR emerged to provide recommendations and guidance on appropriately determining resource requests. Other options, like the container resize policies introduced in Kubernetes 1.27, allow for more fine-grained control over how containers' CPU and memory resources are resized automatically at runtime.

Working with Resource Quotas

The Kubernetes primitive ResourceQuota establishes the usable maximum amount of resources per namespace. Once put in place, the Kubernetes scheduler will take care of enforcing those rules. The following list should give you an idea of the rules that can be defined:

- Setting an upper limit for the number of objects that can be created for a specific type (e.g., a maximum of three Pods)
- Limiting the total sum of compute resources (e.g., 3 Gi of RAM)
- Expecting a Quality of Service (QoS) class for a Pod (e.g., `BestEffort` to indicate that the Pod must not make any memory or CPU limits or requests)

Creating ResourceQuotas

It's usually your task as a Kubernetes administrator to create ResourceQuotas. First, create the namespace the quota should apply to:

```
$ kubectl create namespace team-awesome
namespace/team-awesome created
```

Next, define the ResourceQuota in YAML. To demonstrate the functionality of a ResourceQuota, add constraints to the namespace, as shown in Example 13-4.

Example 13-4. Defining hard resource limits with a ResourceQuota

```
apiVersion: v1
kind: ResourceQuota
```

```
metadata:
  name: awesome-quota
  namespace: team-awesome
spec:
  hard:
    pods: 2                    ❶
    requests.cpu: "1"          ❷
    requests.memory: 1024Mi    ❷
    limits.cpu: "4"            ❸
    limits.memory: 4096Mi      ❸
```

❶ Limit the number of Pods to 2.

❷ Across all Pods in a nonterminal state, the sum of CPU requests cannot exceed this value.

❸ Across all Pods in a nonterminal state, the sum of memory requests cannot exceed this value.

You're ready to create a ResourceQuota for the namespace:

```
$ kubectl create -f awesome-quota.yaml
resourcequota/awesome-quota created
```

Rendering ResourceQuota Details

You can render a table overview of used resources versus hard limits using the `kubectl describe` command:

```
$ kubectl describe resourcequota awesome-quota -n team-awesome
Name:            awesome-quota
Namespace:       team-awesome
Resource         Used  Hard
--------         ----  ----
limits.cpu       0     4
limits.memory    0     4Gi
pods             0     2
requests.cpu     0     1
requests.memory  0     1Gi
```

The `Hard` column lists the same values you provided with the ResourceQuota definition. Those values won't change as long as you don't modify the object's specification. Under the `Used` column, you can find the actual aggregate resource consumption within the namespace. At this time, all values are `0` given that no Pods have been created yet.

Exploring a ResourceQuota's Runtime Behavior

With the quota rules in place for the namespace `team-awesome`, we'll want to see its enforcement in action. We'll start by creating more than the maximum number of Pods, which is two. To test this, we can create Pods with any definition we like. For example, we use a bare-bones definition that runs the image `nginx:1.25.3` in the container, as shown in Example 13-5.

Example 13-5. A Pod without resource requirements

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: team-awesome
spec:
  containers:
  - image: nginx:1.25.3
    name: nginx
```

From that YAML definition stored in *nginx-pod.yaml*, let's create a Pod and see what happens. In fact, Kubernetes will reject the creation of the object with the following error message:

```
$ kubectl apply -f nginx-pod.yaml
Error from server (Forbidden): error when creating "nginx-pod.yaml": \
pods "nginx" is forbidden: failed quota: awesome-quota: must specify \
limits.cpu for: nginx; limits.memory for: nginx; requests.cpu for: \
nginx; requests.memory for: nginx
```

Because we defined minimum and maximum resource quotas for objects in the namespace, we have to ensure that Pod objects actually define resource requests and limits. Modify the initial definition by updating the instruction under `resources`, as shown in Example 13-6.

Example 13-6. A Pod with resource requirements

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx
  namespace: team-awesome
spec:
  containers:
```

```
  - image: nginx:1.25.3
    name: nginx
    resources:
      requests:
        cpu: "0.5"
        memory: "512Mi"
      limits:
        cpu: "1"
        memory: "1024Mi"
```

We should be able to create two uniquely named Pods— `nginx1` and `nginx2` —with that manifest; the combined resource requirements still fit with the boundaries defined in the ResourceQuota:

```
$ kubectl apply -f nginx-pod1.yaml
pod/nginx1 created
$ kubectl apply -f nginx-pod2.yaml
pod/nginx2 created
$ kubectl describe resourcequota awesome-quota -n team-awesome
Name:            awesome-quota
Namespace:       team-awesome
Resource         Used  Hard
--------         ----  ----
limits.cpu       2     4
limits.memory    2Gi   4Gi
pods             2     2
requests.cpu     1     1
requests.memory  1Gi   1Gi
```

You may be able to imagine what would happen if we tried to create another Pod with the definition of `nginx1` and `nginx2`. It will fail for two reasons. The first reason is that we're not allowed to create a third Pod in the namespace, as the maximum number is set to two. The second reason is that we'd exceed the allotted maximum for `requests.cpu` and `requests.memory`. The following error message provides us with this information:

```
$ kubectl apply -f nginx-pod3.yaml
Error from server (Forbidden): error when creating "nginx-pod3.yaml": \
pods "nginx3" is forbidden: exceeded quota: awesome-quota, requested: \
pods=1,requests.cpu=500m,requests.memory=512Mi, used: pods=2,requests.cpu
requests.memory=1Gi, limited: pods=2,requests.cpu=1,requests.memory=1Gi
```

Working with Limit Ranges

In the previous section, you learned how a resource quota can restrict the consumption of resources within a specific namespace in aggregate. For individual Pod objects, the resource quota cannot set any constraints. That's where the limit range comes in. The enforcement of LimitRange rules happens during the admission control phase when processing an API request.

DEFINING MORE THAN ONE LIMITRANGE IN A NAMESPACE

It is best to create only a single LimitRange object per namespace. Default resource requests and limits specified by multiple LimitRange objects in the same namespace causes nondeterministic selection of those rules. Only one of the default definitions will win, but you can't predict which one.

The LimitRange is a Kubernetes primitive that constrains or defaults the resource allocations for specific object types:

- Enforces minimum and maximum compute resource usage per Pod or container in a namespace
- Enforces minimum and maximum storage request per PersistentVolumeClaim in a namespace

- Enforces a ratio between request and limit for a resource in a namespace
- Sets default requests/limits for compute resources in a namespace and automatically injects them into containers at runtime

Creating LimitRanges

The LimitRange offers a list of configurable constraint attributes. All are described in great detail in the Kubernetes API documentation for a LimitRangeSpec. Example 13-7 shows a YAML manifest for a LimitRange that uses some of the constraint attributes.

Example 13-7. A LimitRange defining multiple constraint criteria

```
apiVersion: v1
kind: LimitRange
metadata:
  name: cpu-resource-constraint
spec:
  limits:
```

```
  - type: Container       ❶
    defaultRequest:       ❷
      cpu: 200m
    default:              ❸
      cpu: 200m
    min:                  ❹
      cpu: 100m
    max:                  ❹
      cpu: "2"
```

❶ The context to apply the constraints to (in this case, to a container running in a Pod)

❷ The default CPU resource request value assigned to a container if not provided

❸ The default CPU resource limit value assigned to a container if not provided

❹ The minimum and maximum CPU resource request and limit value assignable to a container

As usual, we can create an object from the manifest with the `kubectl create` or `kubectl apply` command. The definition of the LimitRange has been stored in the file *cpu-resource-constraint-limitrange.yaml*:

```
$ kubectl apply -f cpu-resource-constraint.yaml
limitrange/cpu-resource-constraint created
```

The constraints will be applied automatically when creating new objects. Changing the constraints for an existing LimitRange object won't have any effect on already-running Pods.

Rendering LimitRange Details

Live LimitRange objects can be inspected using the `kubectl describe` command. The following command renders the details of the LimitRange object named `cpu-resource-constraint`:

```
$ kubectl describe limitrange cpu-resource-constraint
Name:       cpu-resource-constraint
Namespace:  default
Type        Resource  Min   Max  Default Request  Default Limit   ...
```

```
----        --------  ---   ---  ---------------  -------------
Container   cpu       100m  2    200m             200m            ...
```

The output of the command renders each limit constraint on a single line. Any constraint attribute that has not been set explicitly by the object will show a dash character (–) as the assigned value.

Exploring a LimitRange's Runtime Behavior

Let's demonstrate what effect the LimitRange has on the creation of Pods. We will walk through two different use cases:

- Automatically setting resource requirements if they have not been provided by the Pod definition
- Preventing the creation of a Pod if the declared resource requirements are forbidden by the LimitRange

Setting default resource requirements

The LimitRange defines a default CPU resource request of 200 m and a default CPU resource limit of 200 m. That means if a Pod is about to be created, and it doesn't define a CPU resource request and limit, the LimitRange will automatically assign the default values.

Example 13-8 shows a Pod definition without resource requirements.

Example 13-8. A Pod defining no resource requirements

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-without-resource-requirements
spec:
  containers:
  - image: nginx:1.25.3
    name: nginx
```

Creating the object from the contents stored in the file *nginx-without-resource-requirements.yaml* will work as expected:

```
$ kubectl apply -f nginx-without-resource-requirements.yaml
pod/nginx-without-resource-requirements created
```

The Pod object will be mutated in two ways. First, the default resource requirements set by the LimitRange are applied. Second, an annotation with the key `kubernetes.io/limit-ranger` will be added that provides meta information on what has been changed. You can find both pieces of information in the output of the `describe` command:

```
$ kubectl describe pod nginx-without-resource-requirements
...
Annotations:      kubernetes.io/limit-ranger: LimitRanger plugin set: cpu
request for container nginx; cpu limit for container nginx
...
Containers:
  nginx:
    ...
    Limits:
      cpu: 200m
    Requests:
      cpu: 200m
...
```

Enforcing resource requirements

The LimitRange can enforce resource limits as well. For the LimitRange object we created earlier, the minimum amount of CPU was set to 100 m, and the maximum amount of CPU was set to 2. To see the enforcement behavior in action, we'll create a new Pod as shown in Example 13-9.

Example 13-9. A Pod defining CPU resource requests and limits

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-with-resource-requirements
spec:
  containers:
  - image: nginx:1.25.3
    name: nginx
    resources:
      requests:
        cpu: "50m"
      limits:
        cpu: "3"
```

The resource requirements of this Pod do not follow the constraints expected by the LimitRange object. The CPU resource request is less than

100 m, and the CPU resource limit is greater than 2. As a result, the object won't be created and an appropriate error message will be rendered:

```
$ kubectl apply -f nginx-with-resource-requirements.yaml
Error from server (Forbidden): error when creating "nginx-with-resource-
requirements.yaml": pods "nginx-with-resource-requirements" is forbidden
[minimum cpu usage per Container is 100 m, but request is 50 m, maximum c
usage per Container is 2, but limit is 3]
```

The error message provides some guidance on expected resource definitions. Unfortunately, the message doesn't point to the name of the LimitRange object enforcing those expectations. Proactively check if a LimitRange object has been created for the namespace and what parameters have been set using `kubectl get limitranges`.

Summary

Resource requests are one of the many factors that the kube-scheduler algorithm considers when making decisions about which node a Pod can be scheduled. A container can specify requests using `spec.containers[].resources.requests`. The scheduler chooses a node based on its available hardware capacity. The resource limits ensure that the container cannot consume more than the allotted resource amounts. Limits can be defined for a container using the attribute `spec.containers[].resources.limits`. Should an application consume more than the allowed amount of resources (e.g., due to a memory leak in the implementation), the container runtime will likely terminate the application process.

A resource quota defines the computing resources (e.g., CPU, RAM, and ephemeral storage) available to a namespace to prevent unbounded consumption by Pods running it. Accordingly, Pods have to work within those resource boundaries by declaring their minimum and maximum resource expectations. You can also limit the number of resource types (like Pods, Secrets, or ConfigMaps) that are allowed to be created. The Kubernetes scheduler will enforce those boundaries upon a request for object creation.

The limit range is different from the ResourceQuota in that it defines resource constraints for a single object of a specific type. It can also help with governance for objects by specifying resource default values that should be applied automatically should the API create request not provide the information.

Exam Essentials

Experience the effects of resource requirements on scheduling and autoscaling

A container defined by a Pod can specify resource requests and limits. Work through scenarios where you define those requirements individually and together for single- and multi-container Pods. Upon creation of the Pod, you should be able to see the effects on scheduling the object on a node. Furthermore, practice how to identify the available resource capacity of a node.

Understand the purpose and runtime effects of resource quotas

A ResourceQuota defines the resource boundaries for objects living within a namespace. The most commonly used boundaries apply to computing resources. Practice defining them, and understand their effect on the creation of Pods. It's important to know the command for listing the hard requirements of a ResourceQuota and the resources currently in use. You will find that a ResourceQuota offers other options. Discover them in more detail for a broader exposure to the topic.

Understand the purpose and runtime effects of limit ranges

A LimitRange can specify resource constraints and defaults of specific primitives. Should you run into a situation where you receive an error message upon creation of an object, check if a LimitRange object enforces those constraints. Unfortunately, the error message does not point out the object that enforces it, so you may have to proactively list LimitRange objects to identify the constraints.

Sample Exercises

Solutions to these exercises are available in Appendix A.

1. You have been tasked with creating a Pod for running an application in a container. During application development, you ran a load test for figuring out the minimum amount of resources needed and the maximum amount of resources the application is allowed to grow to.
   Define those resource requests and limits for the Pod.
   Define a Pod named `hello-world` running the container image `bmuschko/nodejs-hello-world:1.0.0`. The container exposes the port `3000`.

Add a volume of type `emptyDir` and mount it in the container path */var/log*.
For the container, specify the following minimum number of resources:
– CPU: 100 m
– Memory: 500 Mi
– Ephemeral storage: 1 Gi
For the container, specify the following maximum number of resources:
– Memory: 500 Mi
– Ephemeral storage: 2 Gi
Create the Pod from the YAML manifest. Inspect the Pod details. Which node does the Pod run on?

2. In this exercise, you will create a resource quota with specific CPU and memory limits for a new namespace. Pods created in the namespace will have to adhere to those limits.
   Create a ResourceQuota named `app` under the namespace `rq-demo` using the following YAML definition in the file *resourcequota.yaml*:

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: app
spec:
  hard:
    pods: "2"
    requests.cpu: "2"
    requests.memory: 500Mi
```

   Create a new Pod that exceeds the limits of the resource quota requirements, e.g., by defining 1 Gi of memory, but stays below the CPU, e.g., 0.5. Write down the error message.
   Change the request limits to fulfill the requirements to ensure that the Pod can be created successfully. Write down the output of the command that renders the used amount of resources for the namespace.
3. A LimitRange can restrict resource consumption for Pods in a namespace, and assign default computing resources if no resource requirements have been defined. You will practice the effects of a LimitRange on the creation of a Pod in different scenarios.
   Navigate to the directory *app-a/ch13/limitrange* of the checked-out GitHub repository *bmuschko/cka-study-guide*. Inspect the YAML manifest definition in the file *setup.yaml*. Create the objects from the YAML manifest file.

Create a new Pod named `pod-without-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` without any resource requirements. Inspect the Pod details. What resource definitions do you expect to be assigned?

Create a new Pod named `pod-with-more-cpu-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` with a CPU resource request of 400 m and limits of 1.5. What runtime behavior do you expect to see?

Create a new Pod named `pod-with-less-cpu-resource-requirements` in the namespace `d92` that uses the container image `nginx:1.23.4-alpine` with a CPU resource request of 350 m and limits of 400 m. What runtime behavior do you expect to see?